Arabic Dialect Classification Project Report

Razan Ahmed
Dareen Mohamed
Reem Ibrahim

Abstract

This project studies automatic Arabic dialect classification for short social-media text. The task is to classify each Arabic tweet into one of four dialects: Egyptian (E), Gulf (G), Jordanian (J), or Yemeni (Y). The work progressed from a classical TF-IDF + Logistic Regression baseline to recurrent neural networks (LSTM, GRU, and BiLSTM) and finally to MARBERT, an Arabic transformer model. Particular attention was given to data quality: raw duplicate tweets were removed, text was lightly cleaned, cleaned-text conflicts were detected and removed, and the final dataset was split using stratification into training, validation, and test sets. MARBERT produced the strongest validation performance and was then tuned using learning-rate, batch-size, token-length, class-weighted loss, early stopping, warmup, and label-smoothing experiments. The selected MARBERT configuration was finally retrained and evaluated once on the untouched test set, achieving 93.98% accuracy and a macro F1-score of 0.9403.

1 Introduction

Arabic is spoken in many regional varieties whose vocabulary, spelling, expressions, and informal writing conventions can differ substantially. These differences are especially visible in social-media text, where users often write in dialect rather than Modern Standard Arabic. As a result, a natural language processing system trained only on formal Arabic may struggle to correctly interpret dialectal text.

The goal of this project is to build a model that receives an Arabic tweet and predicts one of four dialect classes: Egyptian, Gulf, Jordanian, or Yemeni. Example expressions include:

- Egyptian: عامل إيه النهاردة؟
- Jordanian: شو بتعمل هسا؟
- Gulf: شلونك؟ وش أخبارك؟

Dialect identification can support downstream Arabic NLP applications such as sentiment analysis, conversational systems, content analysis, and dialect-aware text processing. Our project therefore compares multiple modeling approaches, starting with a simple classical baseline and gradually moving toward more powerful neural and transformer-based methods.

2 Project Progress and Experimental Roadmap

The project was developed incrementally rather than by selecting a complex model from the beginning. Our progress can be summarized as follows:

1. Inspect the original dataset and class distribution.

2. Remove duplicate raw tweets and construct a documented cleaning pipeline.
3. Detect duplicates and label conflicts introduced by cleaning, then correct the dataset.
4. Create a stratified 80/10/10 train-validation-test split.
5. Establish a classical TF-IDF + Logistic Regression baseline.
6. Train recurrent neural models: LSTM, LSTM with early stopping, GRU, and BiLSTM.
7. Move to the pretrained Arabic transformer MARBERT after the recurrent models plateaued below 90% validation accuracy.
8. Tune MARBERT using several realistic hyperparameter and training-strategy experiments.
9. Select the best configuration using validation performance only.
10. Retrain the selected configuration in a clean final notebook and evaluate it once on the untouched test set.

This workflow was designed to make each performance gain interpretable and to avoid using the test set during model selection.

3 Dataset

The dataset used in this project was the **Arabic Dialect** dataset published on Kaggle by Wael Shaher [1]. The dataset is available at:

`https://www.kaggle.com/datasets/waelshaher/arabic-dialect`

The file used in our experiments was `Arabic_dialect.csv`. The original dataset contained 38,394 rows and five columns: `Date`, `User`, `Tweet`, `result`, and `t`. The target column `result` contains the dialect label. No missing values were found in the original dataset.

Dialect	**Number of Tweets**
Gulf (G)	12,493
Egyptian (E)	9,891
Jordanian (J)	8,470
Yemeni (Y)	7,540
Total	38,394

Table 1: Original class distribution.

The dataset is moderately imbalanced: Gulf has the most examples, while Yemeni has the fewest. This later motivated a class-weighted MARBERT experiment.

4 Data Cleaning and Preparation

4.1 Raw Duplicate Removal

We first checked exact duplicates in the original `Tweet` column and found 340 repeated tweet texts. Grouping repeated raw tweets by their dialect labels showed no label conflicts at this stage. We therefore removed duplicate raw tweets, reducing the data from 38,394 to 38,054 rows.

4.2 Construction of `clean_tweet`

Instead of relying only on the pre-existing `t` column, we created our own `clean_tweet` field so that the preprocessing steps were explicit and reproducible. The cleaning was intentionally light in order to preserve dialectal cues. It included:

- removing usernames such as `@user`;
- removing URLs;
- collapsing repeated whitespace;
- trimming leading and trailing spaces.

We did not aggressively normalize Arabic spelling because spelling variation can itself contain dialect information.

4.3 Post-cleaning Conflict Detection

An important issue appeared after cleaning: two different original tweets can become identical once usernames and URLs are removed. We therefore grouped rows by `clean_tweet` and counted the number of distinct dialect labels assigned to each cleaned text.

This check found 66 cleaned texts associated with more than one dialect label. These ambiguous cleaned texts were removed entirely. We then removed duplicate `clean_tweet` values and removed any empty strings produced by cleaning. The final corrected dataset contained 35,685 rows, with zero remaining duplicate cleaned tweets.

Stage	**Rows**
Original dataset	38,394
After removing duplicate raw tweets	38,054
After removing empty/conflicting/duplicate cleaned texts	35,685

Table 2: Dataset size through the cleaning process.

4.4 Train, Validation, and Test Split

The corrected dataset was split using stratification so that the four dialects maintained approximately the same proportions in each subset. We used a fixed random seed of 42.

Split	Tweets	Approx. Percentage
Training	28,548	80%
Validation	3,568	10%
Test	3,569	10%

Table 3: Final stratified dataset split.

The validation set was used for model selection and hyperparameter tuning. The test set was intentionally left untouched until the final model had been selected.

5 Evaluation Metrics

We evaluated models primarily using accuracy and macro F1-score. Accuracy measures the overall proportion of correct predictions. Macro F1 computes the F1-score independently for each class and then averages them, giving each dialect equal importance regardless of class size. For the final test run, weighted F1 and per-class precision, recall, and F1 were also reported.

Because the class distribution is not perfectly balanced, macro F1 was used as the main validation selection metric for MARBERT tuning.

6 Baseline and Recurrent Neural Models

6.1 TF-IDF + Logistic Regression

Our first model was a TF-IDF vectorizer followed by Logistic Regression. TF-IDF converts text into sparse numerical features based on term importance, while Logistic Regression learns a decision boundary for the four classes. This model served as a fast and interpretable baseline before moving to neural architectures.

On the corrected validation set, the baseline achieved approximately 85.98% accuracy and a macro F1-score of about 0.86. The confusion matrix showed that Jordanian was particularly difficult and was often confused with Gulf.

6.2 Basic LSTM

The first neural model used a Keras tokenizer and a single LSTM layer. The main configuration was:

- vocabulary size: 20,000;
- maximum sequence length: 50;
- embedding dimension: 128;
- LSTM units: 64;
- dropout: 0.5;
- batch size: 64;
- maximum epochs: 15.

The basic LSTM achieved approximately 84.92% validation accuracy and a macro F1-score of about 0.85. Training accuracy continued to rise while validation performance flattened and validation loss increased, indicating overfitting.

6.3 LSTM with Early Stopping

To reduce overfitting, the LSTM was retrained with early stopping using validation loss and restoration of the best weights. In this run, early stopping did not improve the model: validation accuracy was approximately 84.02% with macro F1 around 0.84. This showed that early stopping is not automatically beneficial when the validation trajectory is noisy or when the architecture requires more time to converge.

6.4 GRU with Early Stopping

A GRU model with the same embedding-based input pipeline was then tested. GRU uses a simpler gating mechanism than LSTM and trained more efficiently in our experiments. It achieved approximately 87.22% validation accuracy and macro F1 around 0.87, clearly improving over the baseline and LSTM variants.

6.5 Bidirectional LSTM

A BiLSTM was tested next. The bidirectional layer processes the sequence in both forward and backward directions, allowing the representation to use context from both sides of each position. The BiLSTM achieved approximately 88.28% validation accuracy and macro F1 around 0.89, making it the strongest recurrent model in the project.

6.6 Comparison Before Transformers

Model	**Validation Accuracy**	**Macro F1**
TF-IDF + Logistic Regression	85.98%	≈ 0.86
Basic LSTM	84.92%	≈ 0.85
LSTM + Early Stopping	84.02%	≈ 0.84
GRU + Early Stopping	87.22%	≈ 0.87
BiLSTM + Early Stopping	88.28%	≈ 0.89

Table 4: Validation performance before introducing MARBERT.

The BiLSTM improved on the baseline, but the performance gap to the later transformer model remained substantial.

7 MARBERT

MARBERT is a deep bidirectional transformer model designed for diverse Arabic varieties [2]. It follows the general BERT-style fine-tuning paradigm [3], where a pretrained transformer is adapted to a downstream classification task by adding a task-specific classification head and training on labeled examples.

For our four-class task, MARBERT was fine-tuned using the pretrained model `UBC-NLP/MARBERT`. Unlike the Keras word-level tokenizer used by the recurrent models, MARBERT uses its pretrained tokenizer and

subword vocabulary. This allows unfamiliar or dialect-specific forms to be decomposed into smaller known units instead of mapping every unseen word to a single out-of-vocabulary token.

7.1 Initial MARBERT Configuration

The reference MARBERT run used:

- learning rate: 2×10^{-5};
- batch size: 16;
- maximum token length: 64;
- epochs: 3;
- weight decay: 0.01;
- validation metric for model selection: macro F1;
- random seed: 42.

This configuration achieved a validation accuracy of approximately 0.940863 (94.09%) and a validation macro F1-score of approximately 0.941698. It therefore clearly outperformed all classical and recurrent models.

Model	**Validation Accuracy**	**Macro F1**
TF-IDF + Logistic Regression	85.98%	≈ 0.86
GRU + Early Stopping	87.22%	≈ 0.87
BiLSTM + Early Stopping	88.28%	≈ 0.89
MARBERT reference run	**94.09%**	**0.9417**

Table 5: Representative comparison between the strongest model families.

8 MARBERT Hyperparameter Tuning and Additional Experiments

After MARBERT became the strongest model, we did not immediately evaluate the test set. Instead, we kept the test set untouched and performed several validation-only experiments. In most experiments, one factor was changed at a time relative to the reference configuration.

8.1 Learning Rate

We tested learning rates of 3×10^{-5} and 1×10^{-5} in addition to the reference 2×10^{-5}. Neither alternative improved validation macro F1. The larger learning rate produced a macro F1 of about 0.9392, while the smaller learning rate produced about 0.9353.

8.2 Batch Size

Batch sizes of 8 and 32 were tested while returning other settings to the reference configuration. Batch size 8 achieved a macro F1 around 0.9390, while batch size 32 achieved about 0.9407. Both remained below the reference value of 0.9417.

8.3 Token-length Analysis

Before continuing to increase the maximum token length, we measured the MARBERT token-length distribution on the training set. The analysis showed:

Statistic	Tokens / Percentage
Mean token length	21.82
Median	18
90th percentile	43
95th percentile	56
99th percentile	76
Maximum	818
Tweets longer than 32 tokens	17.38%
Tweets longer than 48 tokens	7.54%
Tweets longer than 64 tokens	2.49%
Tweets longer than 96 tokens	0.46%
Tweets longer than 128 tokens	0.28%

Table 6: MARBERT token-length analysis on the training set.

Only 2.49% of training tweets exceeded 64 tokens. We therefore tested 96 and 128 as larger alternatives, but neither improved performance. This result supports the practical choice of 64 tokens for this tweet-based dataset.

8.4 Class-weighted Classification

Because the classes were moderately imbalanced, we also tested weighted cross-entropy. The class weights calculated from the training labels were approximately:

- E: 0.9299;
- G: 0.8208;
- J: 1.1218;
- Y: 1.2271.

This gives more influence to mistakes on less frequent classes. The weighted-loss experiment achieved validation accuracy 0.940583 and macro F1 0.941436. This was very close to the reference model but still slightly lower, so the unweighted reference configuration was retained.

8.5 Early Stopping, Warmup, and Label Smoothing

We also explored training-strategy changes rather than only changing basic hyperparameters:

- maximum 5 epochs with early stopping patience 2;
- warmup ratio of 0.10;

- label smoothing factor of 0.05.

The 5-epoch early-stopping experiment performed worse, while warmup and label smoothing were closer to the reference but did not surpass it.

8.6 Complete Tuning Summary

Experiment	Validation Accuracy	Macro F1
Reference: lr 2×10^{-5}, batch 16, length 64	**0.940863**	**0.941698**
Class-weighted loss	0.940583	0.941436
Label smoothing 0.05	0.940303	0.941230
Batch size 32	0.939742	0.940732
Max length 96	0.939462	0.940262
Warmup ratio 0.10	0.938901	0.939948
Max length 128	0.938061	0.939385
Learning rate 3×10^{-5}	0.938061	0.939179
Batch size 8	0.938061	0.939023
5 epochs + early stopping	0.935258	0.936415
Learning rate 1×10^{-5}	0.934137	0.935300

Table 7: MARBERT tuning results. The reference configuration remained the best validation model.

The tuning process therefore produced an important negative result: several reasonable changes were tested, but none consistently improved on the original MARBERT configuration. Rather than selecting a model based on the test set, we kept the configuration with the strongest validation macro F1.

9 Final Model Selection and Retraining

The final configuration was selected entirely from validation performance:

- model: `UBC-NLP/MARBERT`;
- learning rate: 2×10^{-5};
- batch size: 16;
- maximum length: 64;
- epochs: 3;
- weight decay: 0.01;
- seed: 42.

A separate final notebook was created so that the chosen model could be trained and evaluated without rerunning every previous experiment. The selected configuration was retrained from the pretrained MARBERT base model. In this final independent run, validation accuracy was 0.938341 and validation macro F1 was 0.939641. These values were slightly lower than those of the earlier reference run. However, the model configuration had already been selected using validation performance before the test set was evaluated.

After training, the model and tokenizer were saved, and only then was the untouched test set evaluated.

10 Final Test Results

The final MARBERT model was evaluated on 3,569 test tweets exactly once after model selection.

Metric	Final Test Result
Accuracy	0.939759 (93.98%)
Macro F1	0.940271
Weighted F1	0.939658

Table 8: Overall final test performance.

10.1 Per-dialect Performance

Dialect	Precision	Recall	F1-score	Support
Egyptian (E)	0.9394	0.9541	0.9467	959
Gulf (G)	0.9170	0.9558	0.9360	1087
Jordanian (J)	0.9349	0.8844	0.9090	796
Yemeni (Y)	0.9817	0.9574	0.9694	727
Macro average	0.9433	0.9379	0.9403	3569
Weighted average	0.9402	0.9398	0.9397	3569

Table 9: Final MARBERT classification report on the test set.

Yemeni achieved the strongest F1-score (0.9694), while Jordanian was the most difficult class (F1 = 0.9090). This agrees with the earlier baseline experiments, where Jordanian was also frequently confused with Gulf.

10.2 Final Confusion Matrix

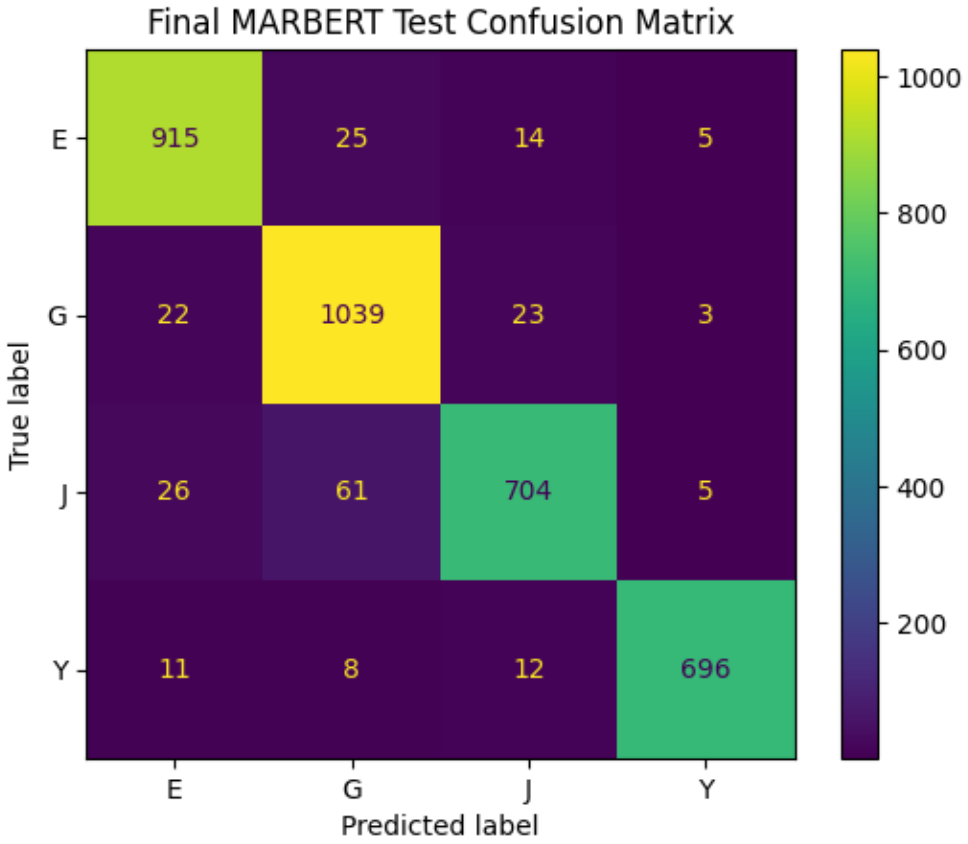


Figure 1: Confusion matrix for the final MARBERT model on the test set.

The final confusion matrix shows 915 correct Egyptian predictions, 1,039 correct Gulf predictions, 704 correct Jordanian predictions, and 696 correct Yemeni predictions. The largest off-diagonal error is Jordanian predicted as Gulf (61 tweets), which explains the lower Jordanian recall compared with the other dialects.

11 Discussion

The experiments show a clear progression in model capability. The classical TF-IDF baseline provided a useful starting point at about 86% validation accuracy. Recurrent models improved only modestly: GRU and BiLSTM were stronger than the baseline, but remained below 89% validation accuracy. MARBERT increased validation accuracy to about 94%, a gain of roughly 5.8 percentage points over the BiLSTM.

Several observations are important:

- **Data quality mattered.** Cleaning created new duplicate strings that were not visible when duplicate checking was performed only on the raw tweet. Detecting and removing 66 cleaned-text label conflicts prevented contradictory examples from leaking into the model.
- **Longer token sequences were unnecessary for most tweets.** The token-length analysis showed that only 2.49% of training tweets exceeded 64 tokens. Increasing the maximum length to 96 or 128 increased computation without improving validation performance.

- **Class weighting was reasonable but not beneficial.** The dataset is imbalanced, but not severely enough for weighted cross-entropy to clearly improve macro F1 in this experiment.
- **More training was not automatically better.** Validation loss often rose while training loss continued to fall, indicating overfitting. The 5-epoch MARBERT experiment with early stopping did not outperform the 3-epoch reference configuration.
- **Transformer pretraining produced the largest gain.** MARBERT's pretrained Arabic representations and subword tokenization were much more effective than learning embeddings from scratch on this dataset.

The final test accuracy (93.98%) was close to the validation performance used during development, suggesting that the selected configuration generalized well to the held-out test data.

12 Limitations

The project has several limitations. First, only four dialect classes were included, so the model should not be interpreted as a complete Arabic dialect identifier. Second, the data consists of tweets, and performance may change on longer text, formal text, or other social-media platforms. Third, the final configuration was evaluated using one final training run rather than repeated training over multiple random seeds. Finally, dialect boundaries can be fuzzy, and some texts may contain mixed or non-dialect-specific language even after cleaning.

These limitations suggest useful future directions, including evaluation on additional dialects and domains, repeated-seed experiments, and targeted investigation of the Jordanian-Gulf confusion pattern.

13 Conclusion

This project developed an Arabic dialect classifier through a sequence of increasingly advanced models. After correcting the data-cleaning pipeline, we established a TF-IDF + Logistic Regression baseline, experimented with several recurrent neural networks, and then fine-tuned MARBERT. MARBERT produced the strongest validation performance and remained the selected model after extensive tuning experiments involving learning rate, batch size, sequence length, class weighting, warmup, early stopping, and label smoothing.

The final selected MARBERT configuration achieved 93.98% accuracy, 0.9403 macro F1, and 0.9397 weighted F1 on the untouched test set. Yemeni was classified most reliably, while Jordanian remained the most challenging dialect. Overall, the project demonstrates that careful data preparation together with a pretrained Arabic transformer provides a substantial improvement over both classical and from-scratch recurrent approaches for this four-dialect tweet classification task.

References

[1] Arabic Dialect Dataset, Kaggle. Dataset used for the Arabic dialect classification experiments.

[2] Muhammad Abdul-Mageed, AbdelRahim Elmadany, and El Moatez Billah Nagoudi. ARBERT & MARBERT: Deep Bidirectional Transformers for Arabic. In *Proceedings of ACL-IJCNLP*, pages 7088–7105, 2021.

[3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[4] Sepp Hochreiter and Jürgen Schmidhuber. Long Short-Term Memory. *Neural Computation*, 9(8):1735–1780, 1997.

[5] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. In *Proceedings of EMNLP*, 2014.

[6] Mike Schuster and Kuldip K. Paliwal. Bidirectional Recurrent Neural Networks. *IEEE Transactions on Signal Processing*, 45(11):2673–2681, 1997.